



PAC2 - Segunda prueba de evaluación continua

Presentación

Después de conocer los sistemas de representación de la información, esta PEC se focaliza en los circuitos lógicos combinacionales. Las funciones lógicas nos permiten describir la funcionalidad de un circuito y mediante las puertas lógicas y los bloques combinacionales podemos implementarlas en un circuito. Todo esto, no se podría hacer sin saber mecanismos de minimización, como mapas de Karnaugh, que nos permiten reducir las dimensiones del circuito que se debe implementar.

Competencias

- Conocer y saber aplicar el álgebra de Boole para la manipulación de funciones lógicas.
- Tener nociones tecnológicas de los circuitos digitales y entender la relación entre los circuitos digitales y las funciones lógicas.
- Conocer y saber utilizar las puertas lógicas y los módulos combinacionales en el diseño de circuitos lógicos.

Objetivos

- Saber aplicar las operaciones básicas y los axiomas del álgebra de Boole.
- Saber representar las funciones lógicas mediante tablas de verdad.
- Saber representar las funciones lógicas mediante expresiones algebraicas.
- Saber analizar un circuito combinacional.
- Saber realizar un cronograma a partir de un circuito digital combinacional.
- Saber sintetizar una función a dos niveles.
- Saber diseñar un circuito combinacional sencillo a partir de los bloques combinacionales de los materiales.

Recursos

Los recursos que se recomienda usar en esta PEC son los siguientes:

Básicos: El módulo 3 de los materiales.

Complementarios: KeMAP, VerilCIRC, VerilCHART y el Wiki de la asignatura.



Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.

Uso de herramientas de IA

En esta actividad no está permitido el uso de herramientas de inteligencia artificial. En el plan docente y en la [web sobre integridad académica y plagio](#) de la UOC encontraréis información sobre que se considera conducta irregular en la evaluación y las consecuencias que puede tener.

Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que librar la solución en un fichero PDF usando una de las plantillas entregadas conjuntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega de la Actividad** correspondiente del apartado **Contenidos** de vuestra aula.
- La fecha tope de entrega es el **6 de noviembre** (a las 24 horas).

Solución de la PEC

Ejercicio 1 [25%]

Dada la tabla de verdad siguiente:

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>f</i>
0	0	0	0	1
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0



- a) [5%] Expresad la función f como suma de minterminos.

La función f expresada como suma de minterminos está formada por el sumatorio de los minterminos de cada posición de la tabla en que f vale 1. Así pues, la función f es:

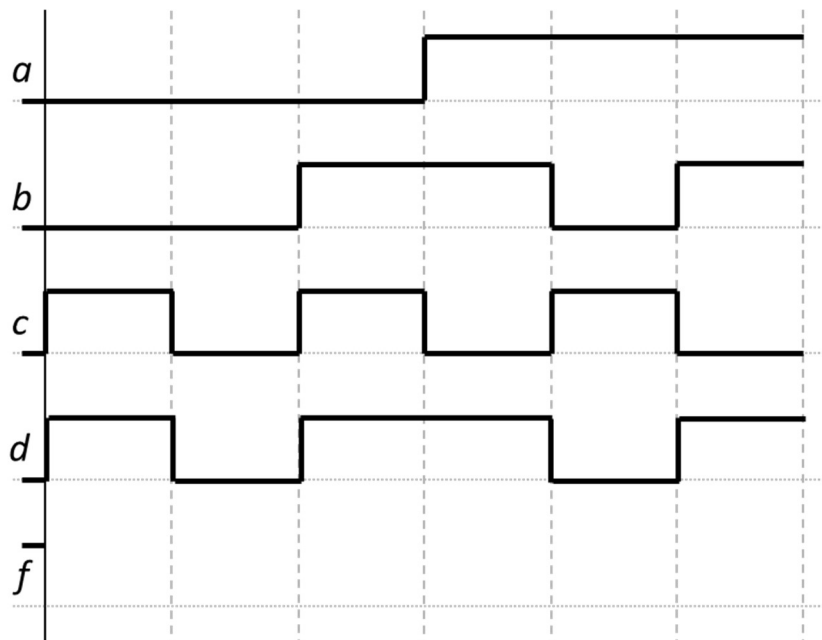
$$f = a'b'c'd' + a'b'c'd + a'bc'd' + a'bcd' + ab'c'd' + ab'c'd + abc'd' + abcd'$$

- b) [10%] Simplificad la función f usando las propiedades y las leyes del álgebra de Boole.

A continuación, se muestra una posible minimización a partir de las leyes del álgebra de Boole. Tened en cuenta que pueden existir varias soluciones según el orden de aplicación de las leyes:

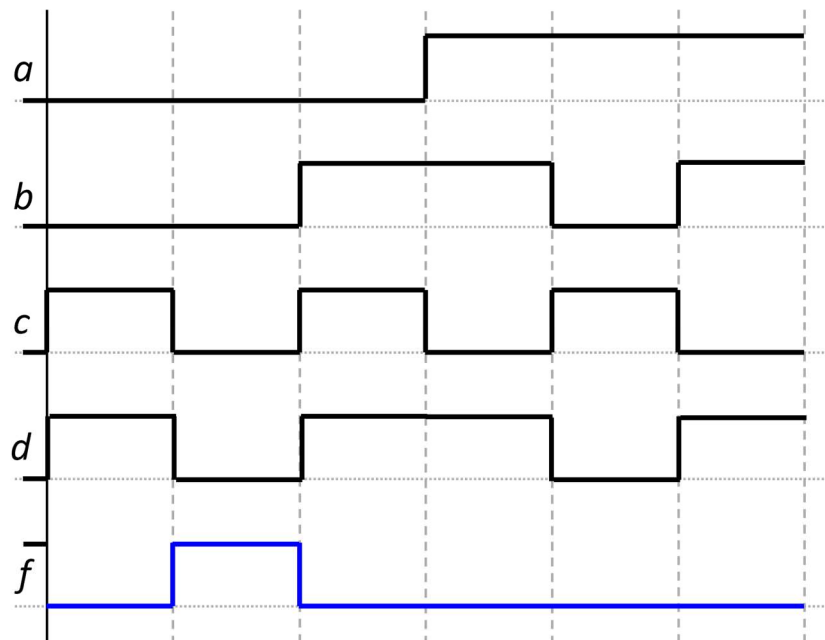
$$\begin{aligned} f &= a'b'c'd' + a'b'c'd + a'bc'd' + a'bcd' + ab'c'd' + ab'c'd + abc'd' + abcd' = \\ &= a'b'c'(d'+d) + a'bd'(c'+c) + ab'c'(d'+d) + abd'(c'+c) && \rightarrow \text{Prop. distributiva} \\ &= a'b'c' + a'bd' + ab'c' + abd' && \rightarrow \text{Complementación} \\ &= (a'+a)b'c' + (a'+a)bd' && \rightarrow \text{Prop. distributiva} \\ &= b'c' + bd' && \rightarrow \text{Complementación} \end{aligned}$$

- c) [10%] Completad el siguiente cronograma:



NOTA: Tenéis disponible el ejercicio en VerilCHART para verificarlo. No hay limitación en el número de intentos.

Vamos buscando en el cronograma las diferentes combinaciones $abcd$ y miramos cuál es la salida f en cada caso. Por ejemplo, en el primer intervalo, vemos que $abcd = 0011$ y, para esta combinación, la salida f vale 0. Por lo tanto, añadimos al cronograma el valor 0. Y así sucesivamente obtenemos el cronograma siguiente:



Ejercicio 2 [25%]

Dada la tabla de verdad siguiente:

<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>g</i>
0	0	0	0	X
0	0	0	1	0
0	0	1	0	1
0	0	1	1	X
0	1	0	0	X
0	1	0	1	0
0	1	1	0	1
0	1	1	1	X
1	0	0	0	X
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	X
1	1	1	0	X
1	1	1	1	1

- a) [15%] Minimizar a dos niveles la función *g* mediante el método de Karnaugh, e implementar el resultado con puertas lógicas a dos niveles.

El mapa de Karnaugh para la función *g* es el siguiente:

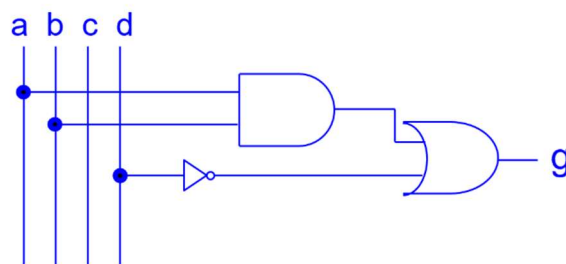


ab \ cd	00	01	11	10
00	X	X	1	X
01	0	0	X	0
11	X	X	1	0
10	1	1	X	1

Y obtenemos esta expresión mínima:

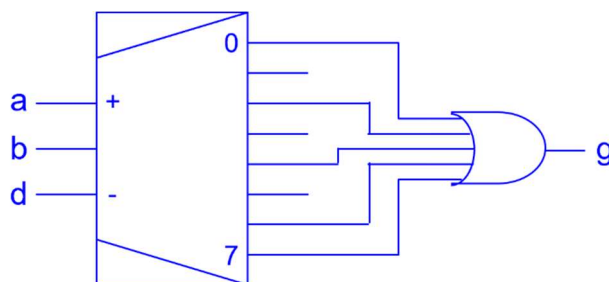
$$g = ab + d'$$

El circuito que la implementa a dos niveles con puertas lógicas es el siguiente:



- b) [10%] Sintetizad la función g con un decodificador 3-8 y las puertas lógicas adicionales que creáis necesarias.

Con un decodificador 4-16 se conectarían directamente las entradas de la función a las entradas del decodificador. En este caso, al ser un decodificador 3-8 y viendo que la función g se puede simplificar con tres variables (a , b y d), conectaremos estas entradas a las entradas del decodificador siguiendo el orden de peso, y conectaremos a una puerta OR las salidas del decodificador correspondientes a las combinaciones por las cuales la salida es 1. Tened en cuenta que las combinaciones *don't care* se pueden conectar opcionalmente, así que para tener un circuito más sencillo y barato no las conectaremos.



NOTA: Tenéis disponible el ejercicio en VerilCIRC y KeMAP para verificarlo. Respecto a KeMAP (para el dos primer apartado del ejercicio), para evitar un mal uso de la herramienta, en caso de que se realicen más de 5 intentos la nota del ejercicio se reducirá al 50%. Os recomendamos hacer pruebas con KeMAP con otros ejercicios antes de hacer el de la PEC. En el caso de VerilCIRC (para el segundo apartado del ejercicio) no hay limitación en el número de intentos.



Ejercicio 3 [25%]

Considerad la existencia del siguiente bloque combinacional MULT22 que multiplica dos números naturales de 2 bits cada uno de ellos y da el resultado en 4 bits:



El rango de los números de entrada A y B es $[0..3]$ y la salida P está dentro del rango $[0..9]$.

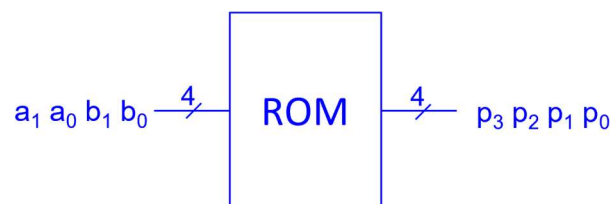
Se pide que:

- a) [5%] Diseñad el circuito MULT22 con una memoria ROM de tamaño 2^4 por 4 bits. Indicad el contenido de la memoria ROM en binario.

El funcionamiento del circuito MULT22 se puede describir con la tabla siguiente:

a_1	a_0		b_1	b_0		p_3	p_2	p_1	p_0
0	0	x	0	0	=	0	0	0	0
0	0	x	0	1	=	0	0	0	0
0	0	x	1	0	=	0	0	0	0
0	0	x	1	1	=	0	0	0	0
0	1	x	0	0	=	0	0	0	0
0	1	x	0	1	=	0	0	0	1
0	1	x	1	0	=	0	0	1	0
0	1	x	1	1	=	0	0	1	1
1	0	x	0	0	=	0	0	0	0
1	0	x	0	1	=	0	0	1	0
1	0	x	1	0	=	0	1	0	0
1	0	x	1	1	=	0	1	1	0
1	1	x	0	0	=	0	0	0	0
1	1	x	0	1	=	0	0	1	1
1	1	x	1	0	=	0	1	1	0
1	1	x	1	1	=	1	0	0	1

Teniendo en cuenta que la ROM tiene las mismas dimensiones que esta tabla, el circuito se puede implementar fácilmente tal y como se muestra a continuación:





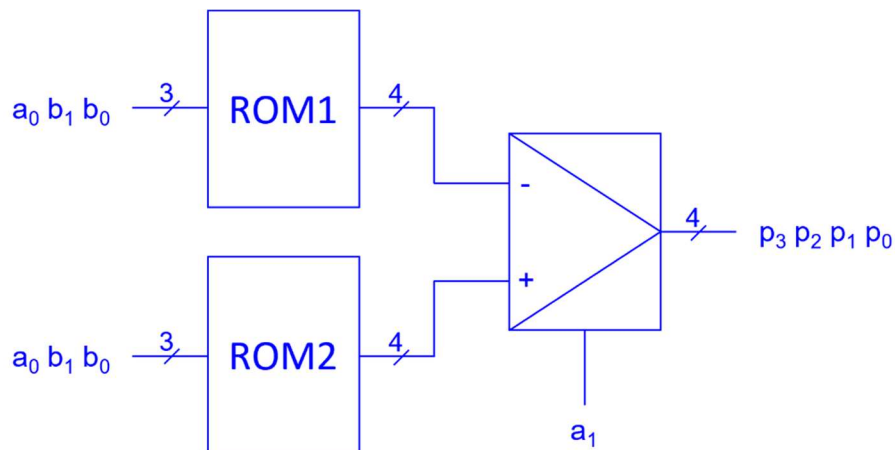
Y su contenido será directamente el de la tabla:

M@	MD
0	0000
1	0000
2	0000
3	0000
4	0000
5	0001
6	0010
7	0011
8	0000
9	0010
10	0100
11	0110
12	0000
13	0011
14	0110
15	1001

- b) [10%] Diseñad el circuito MULT22 a partir de memorias ROM de tamaño 2^3 por 4 bits y las puertas y bloques adicionales que consideréis necesarias. Indicad en binario el contenido de cada una de las memorias ROM utilizadas.

Lo que haremos en este caso es dividir el contenido de la memoria ROM del apartado anterior en dos ROM de 8 palabras de 4 bits cada una ($2^3 \times 4$). En la primera ROM pondremos los ocho primeros valores de la tabla anterior (en los que $a_1 = 0$), y en la segunda ROM pondremos los ocho últimos valores de la tabla anterior (en los que $a_1 = 1$). Entonces, para seleccionar el resultado entre una ROM o la otra, lo hacemos con un multiplexor controlado por a_1 .

El circuito queda tal y como se muestra a continuación:





Y el contenido de las memorias ROM es el siguiente:

ROM1		ROM2	
M@	MD	M@	MD
0	0000	0	0000
1	0000	1	0010
2	0000	2	0100
3	0000	3	0110
4	0000	4	0000
5	0001	5	0011
6	0010	6	0110
7	0011	7	1001

- c) [10%] Los circuitos multiplicadores digitales funcionan tal y como hacemos las multiplicaciones manualmente las personas. Es decir, para multiplicar a_1a_0 x b_1b_0 se hace:

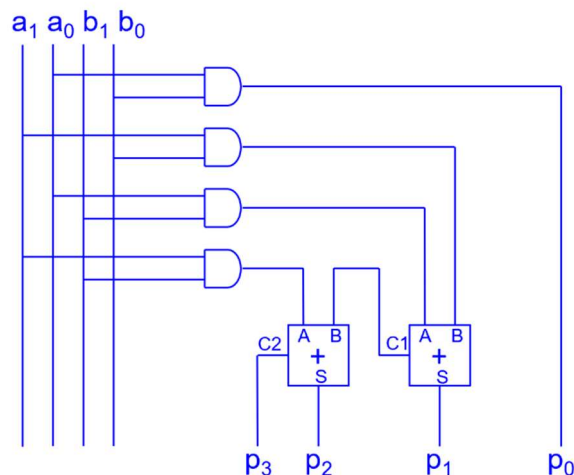
$$\begin{array}{r}
 \begin{array}{cc} & a_1 & a_0 \\ & b_1 & b_0 \end{array} \\
 \hline
 & a_1 \cdot b_0 & a_0 \cdot b_0 \\
 + & a_1 \cdot b_1 & a_0 \cdot b_1 \\
 \hline
 p_3 & p_2 & p_1 & p_0
 \end{array}$$

Siguiendo este razonamiento, diseñad el circuito MULT22 a partir de sumadores de 1 bit y las puertas adicionales que creáis convenientes.

Analizamos cada uno de los dígitos p_3 , p_2 , p_1 y p_0 de la operación:

- $p_0 = a_0 \cdot b_0$
- $p_1 = a_1 \cdot b_0 + a_0 \cdot b_1$ → puede generar acarreo: c_1
- $p_2 = a_1 \cdot b_1 + c_1$ → puede generar acarreo: c_2
- $p_3 = c_2$

Por lo tanto, el circuito que implementa el multiplicador MULT22 es el siguiente:

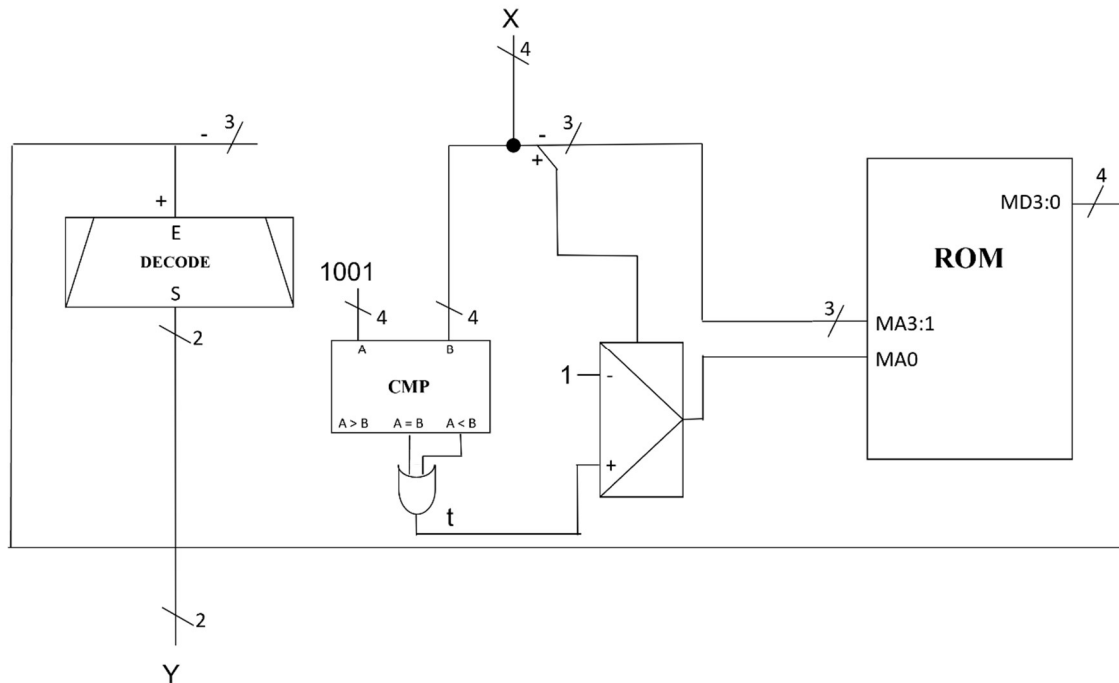


NOTA: Tenéis disponible el ejercicio en VerilCIRC. No hay límite en el número de intentos que hagáis para comprobar vuestra solución.



Ejercicio 4 [25%]

Dado el circuito siguiente:



donde el contenido de cada posición de la memoria ROM viene dado por la expresión:

$$ROM[i] = (i+5) \bmod 16$$

siendo *mod* el operador que da el residuo de la división natural de los dos operandos.

Se pide que rellenéis la tabla de verdad siguiente, que especifica la salida Y en función de la entrada X, calculando previamente los valores intermedios indicados en la tabla.

x_3	x_2	x_1	x_0	t	MA_3	MA_2	MA_1	MA_0	MD_3	MD_2	MD_1	MD_0	y_1	y_0
0	0	0	0											
0	0	0	1											
0	0	1	0											
0	0	1	1											
0	1	0	0											
0	1	0	1											
0	1	1	0											
0	1	1	1											
1	0	0	0											
1	0	0	1											
1	0	1	0											
1	0	1	1											
1	1	0	0											
1	1	0	1											
1	1	1	0											
1	1	1	1											

NOTA: Tenéis disponible el ejercicio en VerilCHART. No hay límite en el número de intentos que hagáis para comprobar vuestra solución.



El circuito recibe como entrada la señal X , de cuatro bits (x_3, x_2, x_1, x_0). El valor de X se compara con el número 9 (1001 en binario) y, en caso de que X sea más grande o igual que 9, la variable t valdrá 1.

El bit de más peso de X , x_3 , también se utiliza para controlar un multiplexor que, según este valor, sacará por la salida el valor 1 o t . Este valor será directamente el bit de menos peso de la entrada de direcciones de una memoria ROM (MA_0). Los otros bits de direcciones son directamente $MA_3 = x_2$, $MA_2 = x_1$ y $MA_1 = x_0$.

La salida de la ROM en la posición i es el valor $i + 5$ (módulo 16). Es decir, la salida de $M[0] = 5$, la de $M[1] = 6$, y así sucesivamente.

Finalmente, de los cuatro bits de salida de la ROM, el bit de más peso se utiliza como entrada en un decodificador 1-2. Cuando este bit vale 0 la salida (y_1, y_0) será 01 y cuando vale 1 la salida será 10.

La tabla queda tal y como se muestra a continuación:

x_3	x_2	x_1	x_0	t	MA_3	MA_2	MA_1	MA_0	MD_3	MD_2	MD_1	MD_0	y_1	y_0
0	0	0	0	0	0	0	0	1	0	1	1	0	0	1
0	0	0	1	0	0	0	1	1	1	0	0	0	1	0
0	0	1	0	0	0	1	0	1	1	0	1	0	1	0
0	0	1	1	0	0	1	1	1	1	1	0	0	1	0
0	1	0	0	0	1	0	0	1	1	1	1	0	1	0
0	1	0	1	0	1	0	1	1	0	0	0	0	0	1
0	1	1	0	0	1	1	0	1	0	0	1	0	0	1
0	1	1	1	0	1	1	1	1	0	1	0	0	0	1
1	0	0	0	0	0	0	0	0	0	1	0	1	0	1
1	0	0	1	1	0	0	1	1	1	0	0	0	1	0
1	0	1	0	1	0	1	0	1	1	0	1	0	1	0
1	0	1	1	1	0	1	1	1	1	1	0	0	1	0
1	1	0	0	1	1	0	0	1	1	1	1	0	1	0
1	1	0	1	1	1	0	1	1	0	0	0	0	0	1
1	1	1	0	1	1	1	0	1	0	0	1	0	0	1
1	1	1	1	1	1	1	1	1	0	1	0	0	0	1